



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de
HONORIS UNITED UNIVERSITIES

RAPPORT DE STAGE D'OBSERVATION PROFESSIONNELLE

Sujet technique

**Conception d'une solution basée sur l'Intelligence Artificielle et
le Machine Learning
pour la supervision proactive des environnements Windows,
VMware et Hyper-V**

Stagiaire	Mehdi L'harti
Entreprise d'accueil	CGI
Encadrant CGI	Nadir Sisah
Encadrant EMSI	Jaouad Tagnamas
Nature du stage	Stage d'observation professionnelle
Période	01/07/2026 – 28/08/2026
Année universitaire	2025-2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué au bon déroulement de mon stage d'observation professionnelle au sein de CGI. Cette immersion sur site m'a permis de découvrir un environnement de travail structuré, exigeant et orienté vers la qualité de service, tout en confrontant mes acquis académiques aux réalités d'un projet informatique à vocation professionnelle.

Je remercie tout particulièrement M. Nadir Sisah, mon encadrant au sein de CGI, pour son accompagnement, ses orientations et les échanges qui m'ont permis de mieux comprendre les attentes d'un contexte professionnel. Son suivi a constitué un repère important dans l'organisation du travail, la prise de recul sur les choix techniques et la progression du projet.

J'adresse également mes sincères remerciements à M. Jaouad Tagnamas, mon encadrant à l'EMSI, pour son encadrement académique, ses recommandations méthodologiques et l'attention portée à la cohérence entre les objectifs du stage, les apprentissages du cycle ingénieur et la rédaction de ce rapport.

Je remercie enfin les équipes de CGI pour l'accueil sur site et pour l'opportunité d'observer les pratiques d'une entreprise internationale de services numériques. Cette expérience a renforcé ma compréhension des exigences liées à l'organisation, à la sécurité, à la documentation, à la qualité logicielle, au travail en équipe et à la validation technique.

Résumé

Ce rapport présente le stage d'observation professionnelle réalisé sur site au sein de CGI Maroc. L'objectif principal du stage était de découvrir le fonctionnement d'un environnement professionnel de services numériques et de mettre en pratique, dans un cadre structuré, des compétences liées au développement logiciel, à la supervision d'infrastructures, au DevOps, à la sécurité et à l'Intelligence Artificielle. Le sujet technique étudié porte sur la conception d'InfraSentinel AI, une plateforme de supervision proactive destinée aux environnements Windows, VMware et Hyper-V.

La solution s'appuie sur un agent Windows, une API Django REST, PostgreSQL, Redis, Celery, un canal WebSocket temps réel et un frontend React/Vite/TypeScript. La couche intelligente combine un moteur de règles, un modèle Isolation Forest pour la détection d'anomalies et une analyse temporelle des tendances afin d'estimer des risques avant qu'un seuil critique ne soit atteint. L'architecture a été validée dans un laboratoire local Docker avec des métriques réelles Windows et des campagnes de tests fonctionnels, de sécurité et de performance.

Les résultats obtenus montrent une plateforme stable dans les scénarios locaux validés : frontend 64/64 tests, WebSocket opérationnel, services Docker sains, télémétrie Windows et GPU réelle, et test de charge jusqu'à 250 agents réalistes avec 0 % d'erreur et un p95 de l'ordre de 98 ms. Les principales limites restantes concernent la validation sur des infrastructures VMware et Hyper-V réelles, l'installation finale du service Windows dans certaines campagnes, l'envoi SMTP externe et la qualification longue durée de la partie prédictive.

Mots-clés : supervision proactive, observabilité, Windows, VMware, Hyper-V, Django, React, PostgreSQL, Redis, Celery, WebSocket, Isolation Forest, Machine Learning, détection d'anomalies, analyse prédictive.

Liste des sigles et abréviations

Sigle	Signification
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
ML	Machine Learning
NOC	Network Operations Center
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
VM	Virtual Machine
VRAM	Video Random Access Memory
WebSocket	Canal bidirectionnel persistant pour les mises à jour temps réel
PFE	Projet / démonstration de fin de parcours académique dans ce contexte
ESN	Entreprise de services numériques

Liste des figures

- Figure 1 - Architecture générale d'InfraSentinel AI
- Figure 2 - Flux d'une mesure depuis la source jusqu'au dashboard
- Figure 3 - Architecture du laboratoire local de validation
- Figure 4 - Gestion des agents Windows
- Figure 5 - Vue globale du dashboard InfraSentinel AI
- Figure 6 - Inventaire des machines et états
- Figure 7 - Fiche détaillée d'une machine Windows réelle
- Figure 8 - Page Anomalies Machine Learning
- Figure 9 - Historique des modèles Isolation Forest
- Figure 10 - Risques prédictifs et tendances
- Figure 11 - Chaîne IA/ML et analyse proactive du risque
- Figure 12 - Alertes et recommandations

Liste des tableaux

- Tableau 1 - Fiche signalétique du stage
- Tableau 2 - Principaux domaines d'activité de CGI Maroc
- Tableau 3 - Objectifs du stage
- Tableau 4 - Besoins fonctionnels et non fonctionnels
- Tableau 5 - Rôles et responsabilités applicatives
- Tableau 6 - Technologies principales
- Tableau 7 - Résultats principaux de performance
- Tableau 8 - Synthèse des validations
- Tableau 9 - Limites et perspectives

Table des matières

Introduction générale	10
Chapitre 1 - Présentation de CGI et cadre professionnel	11
1.1 Présentation générale de CGI	11
1.2 CGI au Maroc et site de Fès	11
1.3 Domaines d'activité et environnement professionnel	11
1.4 Positionnement du stage	12
Chapitre 2 - Déroulement du stage d'observation	13
2.1 Objectifs pédagogiques et professionnels	13
2.2 Méthode de travail et séquences du stage	13
2.3 Pratiques professionnelles observées	13
2.4 Confidentialité et éthique	14
Chapitre 3 - Contexte, problématique et cahier des charges	15
3.1 Contexte de supervision	15
3.2 Problématique	15
3.3 Objectifs de la solution	15
3.4 Besoins fonctionnels et non fonctionnels	15
Chapitre 4 - Conception et architecture d'InfraSentinel AI	17
4.1 Architecture générale	17
4.2 Flux de données	17
4.3 Modèle multi-tenant et RBAC	17
4.4 Architecture locale et conteneurisation	18
Chapitre 5 - Réalisation technique	19
5.1 Agent Windows	19
5.2 Backend et données	19
5.3 Redis, Celery et traitements asynchrones	20
5.4 Frontend et temps réel	20
5.5 VMware et Hyper-V	21
Chapitre 6 - Intelligence Artificielle et supervision proactive	22
6.1 Détection d'anomalies	22
6.2 Isolation Forest	22
6.3 Analyse temporelle et prédiction de risque	23
6.4 Approche hybride règles + ML + tendance	23
6.5 Limites scientifiques	24
Chapitre 7 - Validation, performance et sécurité	25
7.1 Stratégie de validation	25
7.2 Tests fonctionnels et frontend	25
7.3 Tests de charge	25
7.4 Télémétrie CPU/GPU et ML	26
7.5 Sécurité	26
7.6 Bilan de validation	26
Chapitre 8 - Apports du stage, limites et perspectives	28
8.1 Compétences techniques	28
8.2 Compétences professionnelles	28
8.3 Difficultés et enseignements	28

8.4 Perspectives	29
Conclusion générale	30
Bibliographie et webographie	31
Annexes	32

Fiche signalétique du stage

Stagiaire	Mehdi L'harti
Etablissement	EMSI - Ecole Marocaine des Sciences de l'Ingénieur
Formation	Cycle ingénieur - Informatique et Réseaux
Entreprise	CGI Maroc
Lieu	Stage réalisé sur site CGI à Fès - Fès Shore Park
Encadrant professionnel	Nadir Sisah
Encadrant académique	Jaouad Tagnamas
Nature	Stage d'observation professionnelle
Sujet	Conception d'une solution basée sur l'IA et le Machine Learning pour la supervision proactive des environnements Windows, VMware et Hyper-V
Période	A compléter selon la convention

Tableau 1 - Fiche signalétique du stage

Introduction générale

La transformation numérique des organisations repose sur des infrastructures de plus en plus distribuées, virtualisées et dépendantes de services logiciels. Dans ce contexte, la disponibilité des postes Windows, des serveurs, des hyperviseurs et des machines virtuelles devient un enjeu direct de continuité d'activité. Les outils de supervision traditionnels permettent d'observer un état et de déclencher des alertes lorsqu'un seuil est dépassé, mais ils ne fournissent pas toujours une vision unifiée ni une capacité d'anticipation.

Le stage d'observation professionnelle effectué sur site au sein de CGI a constitué une occasion de découvrir le fonctionnement d'une grande entreprise de services numériques tout en mettant en perspective ces problématiques techniques. Au-delà de la réalisation informatique, l'objectif du stage était d'observer la rigueur nécessaire à un projet professionnel : compréhension du besoin, architecture, séparation des responsabilités, sécurité, gestion des erreurs, documentation, tests, mesure des performances et communication des limites.

Le sujet retenu, InfraSentinel AI, vise à concevoir une plateforme centralisée de supervision proactive pour les environnements Windows, VMware et Hyper-V. La plateforme collecte et normalise des métriques, suit l'état des machines, exécute des règles temporelles, détecte des anomalies avec un modèle Isolation Forest et analyse l'évolution des métriques afin d'identifier des risques potentiels. L'objectif n'est donc pas seulement de répondre à la question « que se passe-t-il maintenant ? », mais aussi à « l'évolution actuelle suggère-t-elle qu'un incident pourrait se produire prochainement ? ».

Ce rapport présente d'abord CGI et le cadre professionnel du stage, puis le déroulement de l'observation. Il expose ensuite le contexte et les besoins du projet, l'architecture retenue, les principaux choix techniques, la partie IA/ML, les campagnes de validation et enfin les compétences acquises, les limites et les perspectives.

CHAPITRE 1

Présentation de CGI et cadre professionnel

1.1 Présentation générale de CGI

CGI est une entreprise internationale de services de conseil en technologie de l'information et en management, fondée en 1976. Selon son site institutionnel, le groupe intervient auprès de nombreux secteurs d'activité et s'appuie sur un réseau de plus de 400 sites à l'échelle mondiale. Son positionnement associe conseil, services applicatifs, gestion des infrastructures, transformation numérique et accompagnement des organisations.

L'intérêt d'un stage d'observation au sein d'une telle structure réside dans la possibilité de découvrir une organisation où la technique est indissociable de la qualité de service, de la sécurité, de la relation client, de la gestion des changements et de la documentation. Pour un étudiant en cycle ingénieur, cette immersion permet d'observer comment les connaissances académiques sont transformées en pratiques reproductibles et contrôlées.

1.2 CGI au Maroc et site de Fès

CGI dispose au Maroc de plusieurs implantations, notamment à Casablanca, Rabat et Fès. Le site institutionnel de CGI Maroc indique plus de 1 300 professionnels répartis entre ces implantations et présente les trois sites comme des centres d'excellence mobilisés sur des services applicatifs, la gestion de l'infrastructure et du support, ainsi que des technologies telles que la gestion de données, le CRM, les ERP, les technologies numériques et les environnements historiques.

Le stage a été réalisé sur site CGI à Fès, au Fès Shore Park. Cette présence locale permet d'inscrire l'expérience dans un environnement professionnel réel tout en restant proche de l'écosystème académique régional. L'observation sur site a également permis de mieux percevoir l'importance des règles d'accès, de confidentialité, de communication et de coordination propres à une entreprise de services numériques.

1.3 Domaines d'activité et environnement professionnel

Domaine	Illustration dans le contexte du stage
Services applicatifs	Conception, développement, intégration, maintenance et validation de solutions logicielles.
Infrastructure et support	Supervision, disponibilité, suivi des performances, traitement des incidents et continuité de service.
Data et IA	Exploitation des données techniques, détection d'anomalies, analyse de tendances et automatisation.
Cybersécurité et qualité	Contrôle d'accès, gestion des secrets, audit, principe du moindre privilège, validation et traçabilité.
Conseil et transformation numérique	Traduction d'un besoin métier ou opérationnel en architecture, processus et solution mesurable.

Tableau 2 - Principaux domaines d'activité observés ou reliés au contexte du stage

Le rapport ne décrit volontairement aucun projet client, aucune donnée interne ni aucun processus confidentiel de CGI. Les éléments présentés concernent uniquement les informations publiques de l'entreprise, les apprentissages généraux du stage et le projet technique InfraSentinel AI.

1.4 Positionnement du stage

Le stage combine deux dimensions complémentaires. La première est une dimension d'observation professionnelle : comprendre le niveau d'exigence d'une organisation structurée, le rôle de la documentation, la validation avant livraison, la gestion des accès et la nécessité de communiquer clairement un résultat technique. La seconde est une dimension d'application : concevoir et valider une solution de supervision capable d'illustrer ces principes.

Positionnement du sujet

InfraSentinel AI constitue un support technique de mise en pratique. Le rapport distingue systématiquement ce qui a été validé sur une machine réelle, ce qui a été testé dans un laboratoire contrôlé et ce qui reste non testé faute d'environnement externe autorisé.

CHAPITRE 2

Déroulement du stage d'observation

2.1 Objectifs pédagogiques et professionnels

Objectif	Résultat attendu
Découvrir l'environnement professionnel	Comprendre l'organisation générale d'une ESN, la rigueur des échanges et la place de la qualité.
Relier théorie et pratique	Mettre en œuvre architecture logicielle, réseaux, bases de données, systèmes et IA dans un projet cohérent.
Améliorer l'autonomie	Savoir auditer, diagnostiquer, corriger, tester et documenter une solution.
Développer la communication technique	Présenter une architecture, un résultat, une limite et une recommandation de manière claire.
Adopter une démarche d'ingénieur	Mesurer plutôt qu'affirmer, préserver la sécurité et distinguer preuve réelle, simulation et hypothèse.

Tableau 3 - Objectifs du stage

2.2 Méthode de travail et séquences du stage

Le travail a été organisé de manière progressive afin de limiter les régressions et de conserver une trace des décisions. Plutôt que de modifier l'ensemble du projet en une seule étape, la démarche a consisté à établir un état de référence, sécuriser l'environnement, valider chaque composant puis augmenter progressivement le niveau de réalisme des tests.

1. Audit initial du dépôt, de l'architecture, des dépendances, des scripts et des tests.
2. Nettoyage des dépendances cloud inutiles et définition d'un laboratoire local reproductible.
3. Validation de PostgreSQL, Redis, Celery, WebSocket, du backend et du frontend.
4. Validation d'un agent Windows réel et de la remontée de métriques.
5. Renforcement de la détection d'anomalies, des tendances prédictives et des règles temporelles.
6. Campagnes de charge, stress CPU/GPU, sécurité, résilience et sauvegarde/restauration.
7. Reconstruction et amélioration finale du frontend, documentation et préparation de démonstration.

2.3 Pratiques professionnelles observées

Plusieurs principes professionnels ont marqué le déroulement du stage. Le premier est l'importance de la traçabilité : un résultat doit être reproductible et appuyé par un rapport, une commande, un test ou une mesure. Le deuxième est la séparation entre développement et validation : le fait qu'une fonctionnalité existe dans le code ne suffit pas à conclure qu'elle fonctionne dans un environnement réel.

Le stage a également renforcé l'importance de la gestion de configuration et du contrôle de version. Les modifications significatives ont été associées à des checkpoints Git, des branches dédiées et des rapports de validation. Cette discipline permet de revenir à un état stable, de comparer les évolutions et de limiter l'impact d'une régression.

Enfin, la sécurité et la confidentialité ont été traitées comme des contraintes transversales. Les secrets doivent être injectés par environnement, les tokens ne doivent pas être exposés dans l'interface, les accès doivent être contrôlés côté serveur, et les tests doivent être exécutés uniquement sur des équipements autorisés.

2.4 Confidentialité et éthique

Dans une entreprise de services numériques, l'accès aux informations et aux infrastructures doit être proportionné au besoin. Dans le cadre de ce stage, les tests ont été limités aux machines, services et environnements autorisés. Les scénarios de stress ont été bornés dans le temps et accompagnés de seuils de sécurité afin d'éviter une dégradation matérielle ou une perturbation extérieure.

Cette approche est également importante pour la rédaction du rapport. Aucune donnée client, aucun mot de passe, aucun token agent, aucune adresse interne sensible ni aucune information opérationnelle confidentielle n'est présentée. Les captures intégrées ont été vérifiées afin de masquer les identifiants ou données qui n'ont pas vocation à être diffusés.

CHAPITRE 3

Contexte, problématique et cahier des charges

3.1 Contexte de supervision

Une infrastructure informatique moderne peut réunir postes Windows, serveurs physiques, machines virtuelles, hyperviseurs, services applicatifs, bases de données et composants réseau. Chacun produit des métriques différentes : charge CPU, mémoire, stockage, débit réseau, latence, disponibilité, état des services et événements. Lorsque ces données sont dispersées, il devient difficile d'obtenir une vue cohérente de la santé globale.

La virtualisation renforce cette difficulté. VMware et Hyper-V possèdent leurs propres mécanismes d'administration et de collecte, tandis que Windows expose des informations locales sur le système et le matériel. Une plateforme centralisée doit donc normaliser les données avant de pouvoir appliquer des règles ou des modèles d'analyse communs.

3.2 Problématique

Problématique générale

Comment concevoir une plateforme sécurisée et multi-environnement capable de centraliser des métriques réelles Windows, VMware et Hyper-V, de détecter automatiquement les comportements anormaux et d'identifier des tendances susceptibles d'annoncer un incident avant qu'il ne devienne critique ?

La problématique implique trois niveaux d'analyse. La supervision classique doit d'abord détecter un dépassement de seuil ou une indisponibilité. Le Machine Learning doit ensuite repérer des combinaisons de métriques inhabituelles qui ne sont pas nécessairement capturées par une règle simple. Enfin, l'analyse temporelle doit mesurer une évolution afin d'estimer qu'une ressource se rapproche d'un état dangereux.

3.3 Objectifs de la solution

- Centraliser l'état des machines et environnements supervisés.
- Collecter des métriques Windows réelles via un agent authentifié.
- Prévoir des connecteurs VMware/vCenter et Hyper-V.
- Normaliser les métriques et les unités avant stockage.
- Déclencher des règles basées sur seuil, durée, sévérité et récupération.
- Détecter des anomalies multidimensionnelles avec Isolation Forest.
- Analyser les tendances et estimer un risque de franchissement de seuil.
- Afficher alertes, anomalies, prédictions et recommandations dans un dashboard temps réel.
- Garantir RBAC, isolation multi-tenant, audit, sécurité des tokens et déduplication.
- Valider la solution par tests automatisés, tests de charge, scénarios de résilience et mesures.

3.4 Besoins fonctionnels et non fonctionnels

Catégorie	Besoins principaux
Supervision	Machines, agents, états ONLINE/DEGRADED/OFFLINE, CPU, RAM, disque, réseau, services, GPU si disponible.
Virtualisation	Découverte et collecte VMware/vCenter ; collecte Hyper-V via PowerShell/worker Windows.
IA/ML	Entraînement versionné, inférence, score d'anomalie, stabilité temporelle, tendances et risque.
Alertes	Création, sévérité, déduplication, acquittement, résolution, recommandations et notifications.
Temps réel	WebSocket authentifié, reconnexion, replay borné et mise à jour du dashboard sans rechargement.
Sécurité	JWT, RBAC, multi-tenant, tokens agents hashés, CSRF/CSP, secrets hors dépôt, audit append-only.
Qualité	Tests automatisés, lint, build, audit de dépendances, Docker healthchecks, sauvegarde/restauration.
Performance	Support d'un nombre croissant d'agents, suivi p50/p95/p99, erreurs et saturation des services.

Tableau 4 - Besoins fonctionnels et non fonctionnels

CHAPITRE 4

Conception et architecture d'InfraSentinel AI

4.1 Architecture générale

L'architecture retenue sépare les sources de données, le traitement serveur, la persistance, l'analyse et la présentation. Cette séparation rend le système plus facile à tester et limite le couplage entre l'agent Windows, les connecteurs de virtualisation et le frontend.

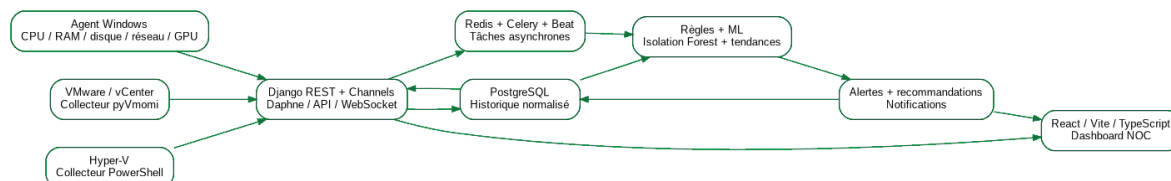


Figure 1 - Architecture générale d'InfraSentinel AI

Le backend constitue le point central de confiance. Le client ou l'agent ne décide jamais du tenant auquel une donnée appartient. Les relations entre utilisateur, customer, environnement, machine et agent sont revalidées côté serveur. Les événements significatifs sont persistés avant leur diffusion temps réel, ce qui permet un mécanisme de replay contrôlé après reconnexion.

4.2 Flux de données



Figure 2 - Flux d'une mesure depuis la source jusqu'au dashboard

Une mesure est d'abord authentifiée et rattachée à une source connue. Elle est ensuite normalisée afin de garantir des noms de métriques et des unités cohérents. Après persistance dans PostgreSQL, elle peut être utilisée par le moteur de règles, les traitements ML et les agrégations. Si un événement doit être exposé à l'utilisateur, il est publié via le mécanisme temps réel après validation de la transaction.

4.3 Modèle multi-tenant et RBAC

InfraSentinel AI a été conçu pour plusieurs rôles. L'administrateur possède une vision globale autorisée, tandis qu'un client ne voit que les machines et événements de son périmètre. Cette séparation est appliquée dans les querysets et permissions serveur, et non uniquement dans la barre de navigation du frontend.

Rôle	Responsabilité fonctionnelle
ADMIN	Administration du tenant, utilisateurs, agents, règles et vue globale autorisée.
SUPERVISOR	Supervision et opérations autorisées sur le périmètre attribué.
TECHNICIAN	Consultation technique et actions opérationnelles limitées selon les permissions.

Rôle	Responsabilité fonctionnelle
CLIENT	Accès uniquement à ses propres assets, métriques, alertes et analyses.
VIEWER	Lecture seule lorsqu'elle est activée par la configuration.

Tableau 5 - Rôles et responsabilités applicatives

4.4 Architecture locale et conteneurisation

Après l'abandon des expérimentations dépendantes de fournisseurs cloud, le projet a été consolidé dans un laboratoire local reproductible. Docker Compose orchestre PostgreSQL, Redis, l'API Django/Daphne, Celery Worker, Celery Beat et le frontend Nginx. Cette approche permet de tester l'ensemble de la chaîne sans coût d'hébergement et de conserver un environnement proche d'un déploiement professionnel.

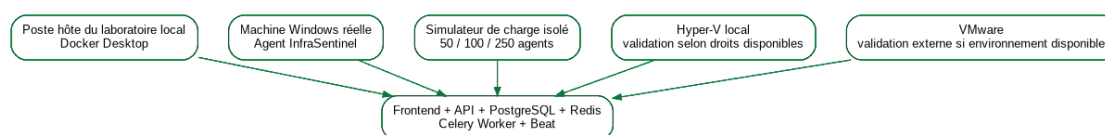


Figure 3 - Architecture du laboratoire local de validation

Les agents simulés sont utilisés uniquement pour la charge. Ils sont isolés des données opérationnelles afin de ne pas être présentés comme de vraies machines. Cette distinction REAL / CONTROLLED_TEST / SIMULATED fait partie des garanties de cohérence de la démonstration.

CHAPITRE 5

Réalisation technique

5.1 Agent Windows

L'agent Windows est écrit en Python. Il collecte des métriques système avec une logique de heartbeat et d'envoi authentifié vers l'API. Les informations couvertes comprennent notamment CPU, mémoire, stockage, entrées/sorties, réseau, uptime, processus, services critiques et, lorsqu'une carte NVIDIA est disponible, utilisation GPU, mémoire vidéo et température.

L'identité de chaque agent est indépendante. Le serveur stocke le hash du token, permet la révocation et la rotation, et rattache l'agent à une machine et à un tenant. Le spool local Windows peut protéger les données en attente par DPAPI. L'agent refuse les redirections inattendues et impose HTTPS hors localhost lorsqu'il est configuré en mode sécurisé.



Figure 4 - Gestion des agents Windows

5.2 Backend et données

Le backend repose sur Django 6, Django REST Framework, SimpleJWT, Channels et Daphne. PostgreSQL constitue la base de référence. Les domaines métiers sont séparés entre comptes, inventaire, métriques, monitoring, moteur ML, intégrations, notifications, temps réel et tâches asynchrones. Cette structuration facilite les tests et la maîtrise des responsabilités.

Composant	Technologie / rôle
Backend API	Django 6.0.8, Django REST Framework 3.17.1, SimpleJWT
Temps réel	Django Channels 4.3.2, Daphne 4.2.3, channels-redis
Base de données	PostgreSQL 17 avec psycopg et pool de connexions
Asynchrone	Celery 5.6 + Redis, Celery Beat
Machine Learning	scikit-learn 1.9, Isolation Forest, feature engineering et tendances

Composant	Technologie / rôle
VMware	pyVmmomi 9
Hyper-V	PowerShell avec worker Windows/permissions adaptées
Frontend	React 19, TypeScript 5.9, Vite, TanStack Query, Recharts, Axios
Conteneurisation	Docker Compose, Nginx; Caddy pour TLS dans l'overlay de production

Tableau 6 - Technologies principales

5.3 Redis, Celery et traitements asynchrones

Redis est utilisé pour la communication asynchrone et le canal de diffusion temps réel. Celery exécute les traitements qui ne doivent pas bloquer une requête HTTP : collectes externes, évaluations de règles, analyses ML, notifications, agrégations et rapports. Celery Beat publie les tâches planifiées.

L'idempotence est importante pour éviter qu'une tâche rejouée crée plusieurs alertes ou plusieurs métriques. Le projet associe des clés métiers aux opérations critiques et conserve un historique des exécutions. La campagne de résilience a également testé l'arrêt puis le redémarrage de Redis, du worker, de Beat, de l'API et de PostgreSQL.

5.4 Frontend et temps réel

Le frontend final a été reconstruit en React, Vite et TypeScript avec une architecture par domaines, un client API centralisé, TanStack Query pour le cache et Recharts pour les visualisations. L'interface reprend les codes d'un outil NOC : état global, machines nécessitant une attention, alertes critiques, anomalies ML, risques prédictifs et activité récente.

Le WebSocket permet de mettre à jour les états importants sans recharger la page. La logique gère l'obtention d'un ticket, le handshake, la reconnexion, le replay limité et la déduplication. Le statut temps réel est visible dans l'interface afin de distinguer un écran à jour d'une connexion interrompue.

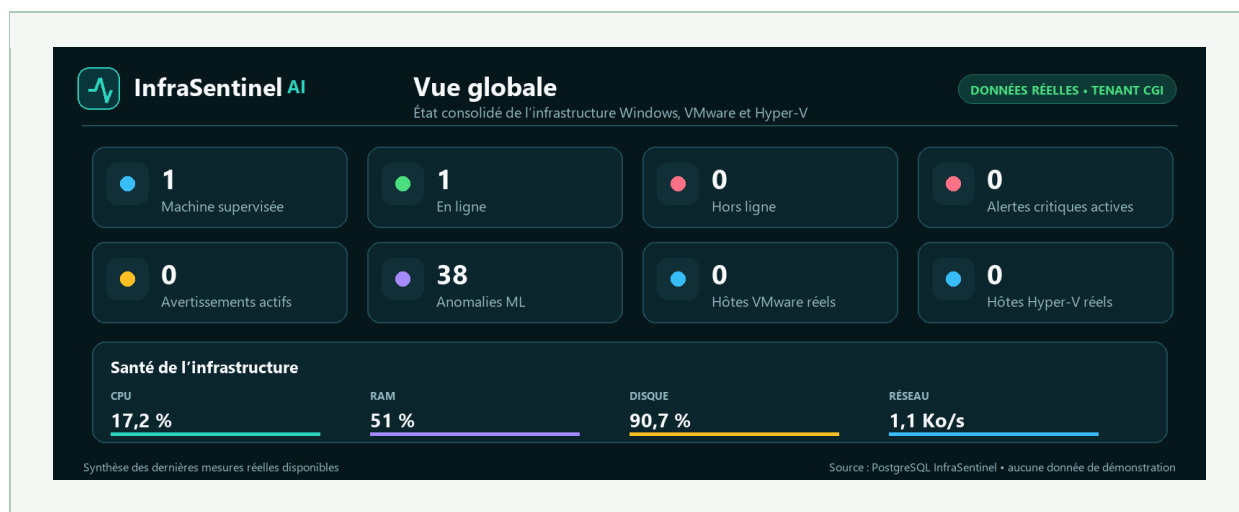


Figure 5 - Vue globale du dashboard InfraSentinel AI

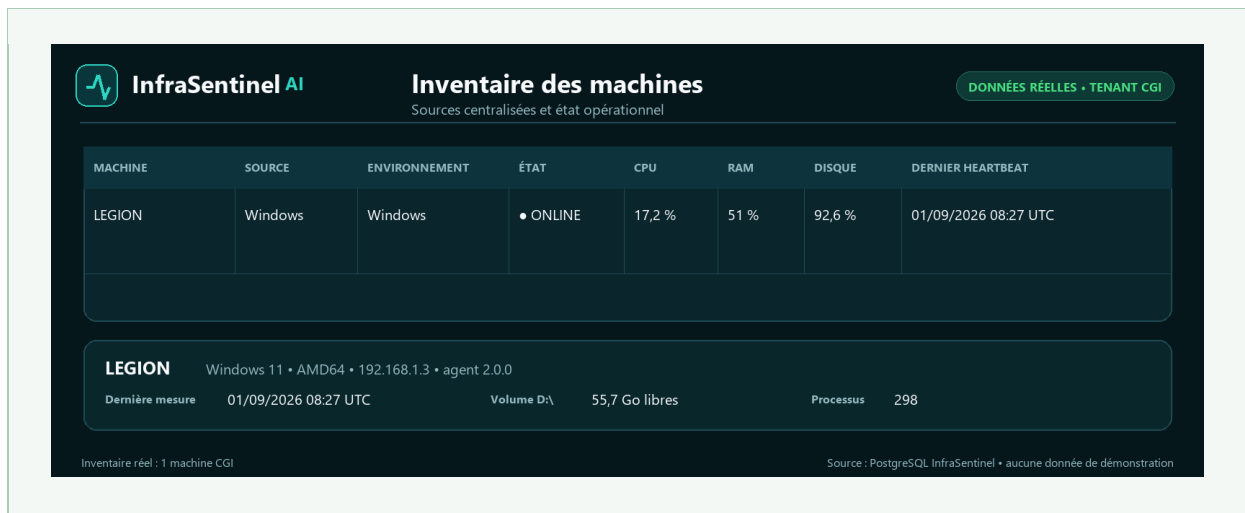


Figure 6 - Inventaire des machines et états

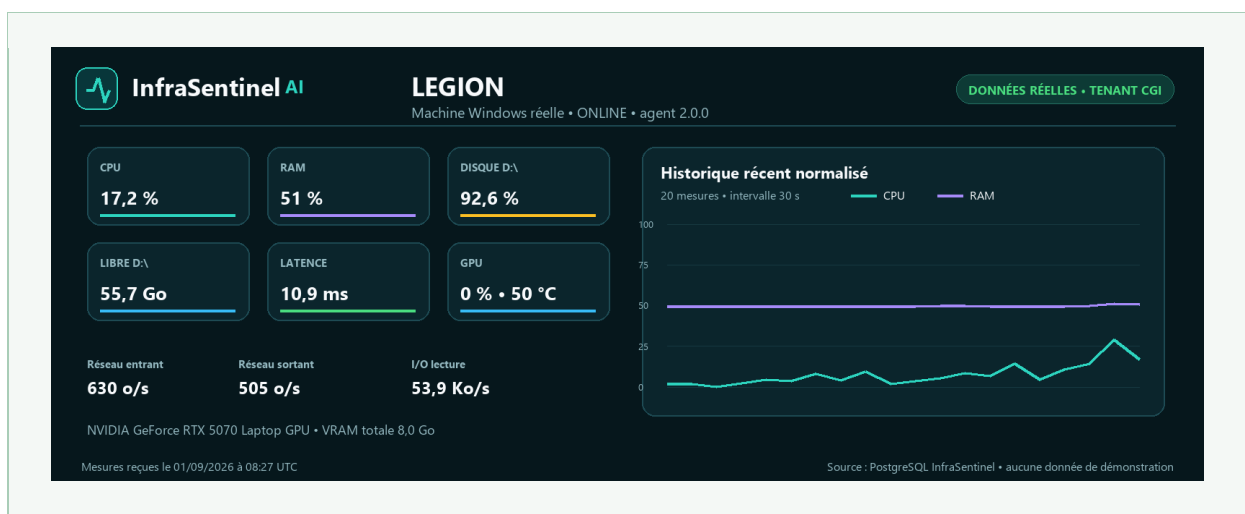


Figure 7 - Fiche détaillée d'une machine Windows réelle

5.5 VMware et Hyper-V

Deux connecteurs complètent l'agent Windows. La partie VMware s'appuie sur pyVmmomi afin de découvrir les hôtes et machines virtuelles vSphere et de normaliser leurs métriques. La partie Hyper-V repose sur PowerShell et nécessite un worker Windows disposant du module Hyper-V ainsi que des permissions adaptées.

Ces composants sont présents, testés avec des mocks et intégrés aux API et au frontend. Toutefois, le rapport conserve une distinction stricte entre implémentation et preuve réelle : la campagne locale ne disposait pas d'un vCenter autorisé, et l'appel Get-VM a été limité par les privilèges disponibles. Les validations VMware et Hyper-V réelles restent donc à compléter sur une infrastructure autorisée.

CHAPITRE 6

Intelligence Artificielle et supervision proactive

6.1 Détection d'anomalies

La valeur ajoutée du sujet ne réside pas uniquement dans l'affichage de métriques. Une plateforme proactive doit repérer qu'un comportement s'éloigne de la normale et fournir une information suffisamment tôt pour permettre une action préventive. Pour cela, InfraSentinel AI construit des fenêtres de métriques historiques et applique un modèle non supervisé.

6.2 Isolation Forest

Isolation Forest est adapté à la détection d'anomalies lorsque les exemples explicitement étiquetés « panne » sont rares. Le principe consiste à isoler plus rapidement les observations atypiques que les observations appartenant à une zone dense du comportement habituel. Le modèle produit donc un score d'anomalie qu'il faut interpréter dans le contexte des features utilisées et du seuil de décision.

Le paramètre de contamination ne représente pas une probabilité de panne. Il sert à calibrer la proportion attendue d'observations atypiques lors de l'entraînement. La qualité du modèle dépend surtout de la représentativité du comportement normal, du volume de fenêtres, du choix des features et de la stabilité de la politique d'alerte.

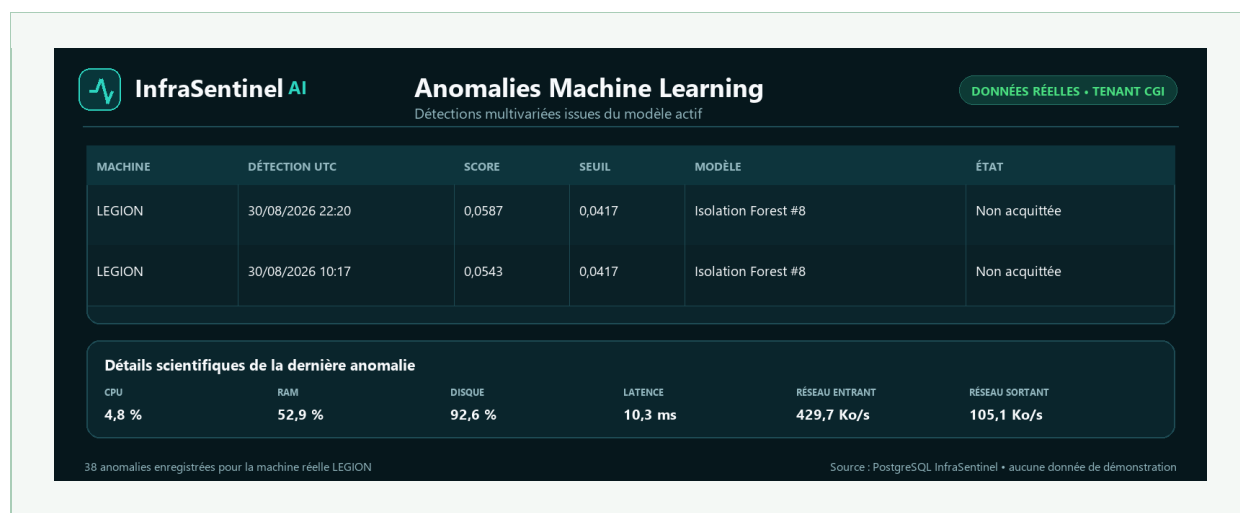


Figure 8 - Page Anomalies Machine Learning

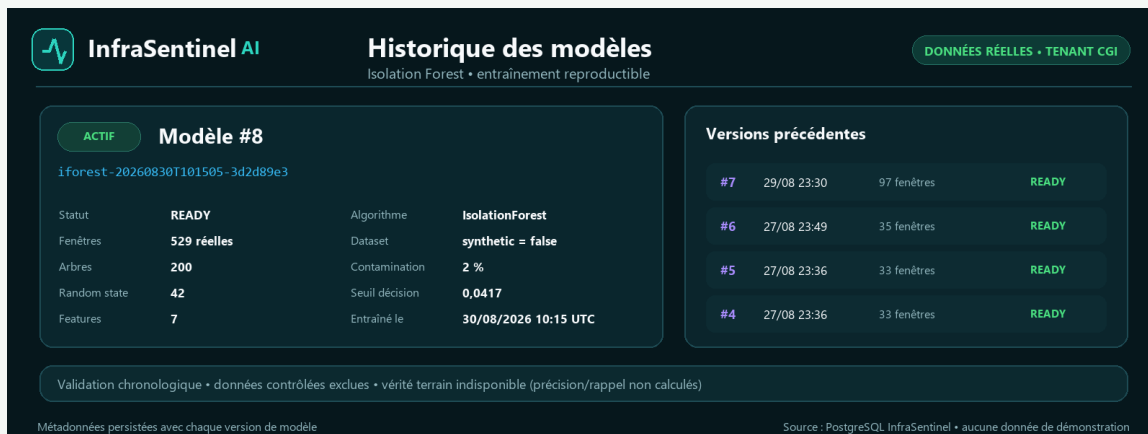


Figure 9 - Historique des modèles Isolation Forest

6.3 Analyse temporelle et prédiction de risque

Isolation Forest détecte principalement une situation inhabituelle ; il ne prédit pas à lui seul l'heure exacte d'une panne. Pour rendre la supervision réellement proactive, InfraSentinel complète l'inférence par une analyse temporelle : pente, vitesse d'évolution, persistance et estimation du temps avant franchissement d'un seuil lorsque le calcul est pertinent.

Une série de valeurs de RAM passant par exemple de 52 % à 58 %, 64 %, 71 % puis 79 % peut produire une tendance croissante avant que la règle critique ne se déclenche. Le système peut alors présenter un risque élevé et une estimation prudente du franchissement potentiel, tout en affichant « historique insuffisant » lorsqu'il ne dispose pas de données suffisantes.



Figure 10 - Risques prédictifs et tendances

6.4 Approche hybride règles + ML + tendance

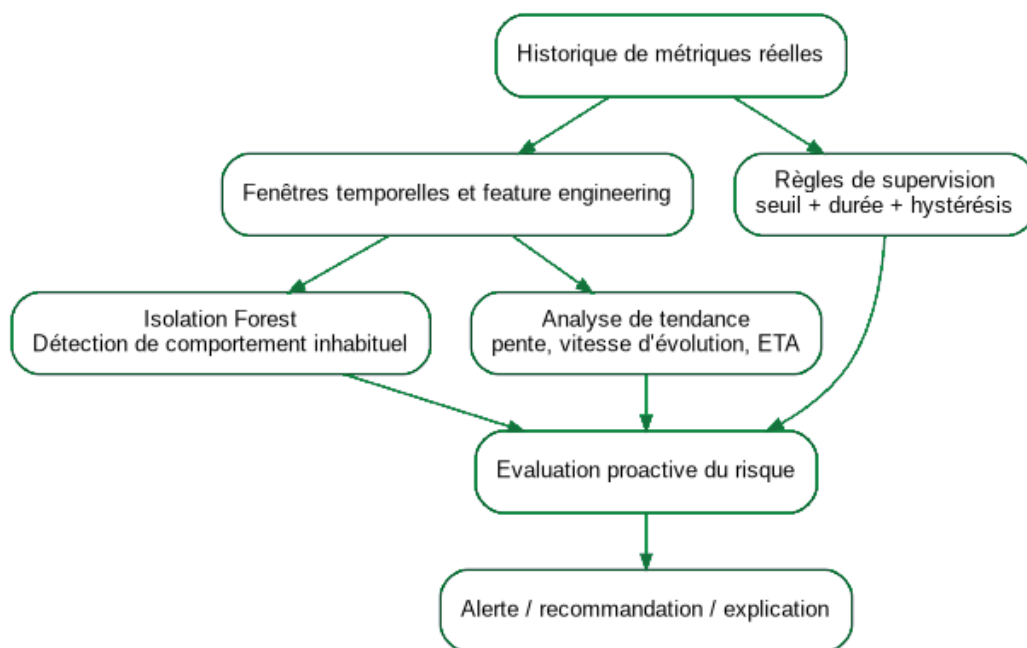


Figure 11 - Chaîne IA/ML et analyse proactive du risque

L'approche hybride permet de conserver l'explicabilité des règles tout en ajoutant une capacité de détection plus souple. Une règle répond à une condition définie à l'avance ; l'Isolation Forest recherche un comportement inhabituel ; la tendance mesure l'évolution. L'évaluation finale du risque agrège ces signaux sans prétendre fournir une certitude de panne.

Pour réduire les faux positifs, le moteur ML n'escalade pas nécessairement une alerte à partir d'une seule fenêtre. Des critères de persistance et d'hystérésis ont été introduits afin de demander plusieurs fenêtres anormales avant escalation et plusieurs fenêtres normales avant résolution. Cette stabilité temporelle est essentielle dans un système de supervision réel.

6.5 Limites scientifiques

La validation scientifique d'un modèle d'anomalies est plus difficile qu'un simple test logiciel. Pour mesurer précisément précision, rappel, F1, faux positifs et faux négatifs, il faut disposer d'une vérité terrain correctement étiquetée. Les campagnes du stage ont renforcé le nombre de fenêtres normales et vérifié des anomalies contrôlées, mais elles ne constituent pas encore un jeu de données industriel long terme.

La bonne pratique retenue est donc de présenter séparément : une détection sur métriques réelles, un scénario contrôlé servant à reproduire une anomalie et les limites restantes. Cette transparence est préférable à l'affichage de scores artificiellement parfaits.

CHAPITRE 7

Validation, performance et sécurité

7.1 Stratégie de validation

La validation a été organisée selon plusieurs niveaux : tests unitaires, tests d'intégration, build, healthchecks Docker, test d'agent réel, tests réseau/WebSocket, scénarios de charge, stress matériel contrôlé, résilience et audit de sécurité. Les résultats sont classés en PASS, PARTIAL, FAIL ou NOT TESTED afin d'éviter de confondre l'existence d'un composant et sa validation dans une infrastructure réelle.

7.2 Tests fonctionnels et frontend

La reconstruction finale du frontend a été accompagnée d'une suite de 64 tests répartis sur 17 fichiers. Tous ont réussi, sans skip ni échec. TypeScript, ESLint et Prettier sont passés sans erreur ; le build local et le build Docker ont également réussi. L'audit npm n'a détecté aucune vulnérabilité connue au moment de la campagne.

La validation a couvert les routes principales, les états de chargement/erreur, la pagination prédictive, la déconnexion mobile, le focus clavier, le cache multi-tenant, le WebSocket, les unités lisibles et la politique CSP same-origin. Cette campagne permet de considérer le frontend comme prêt pour une démonstration PFE basée sur la machine Windows réelle, avec des réserves sur certains audits spécialisés et intégrations externes.

7.3 Tests de charge

Profil	Agents	Débit	p95	Erreurs	Connexions PG max
Réaliste 30 s	50	2,80 req/s	71,6 ms	0 %	16
Réaliste 30 s	100	5,60 req/s	76,2 ms	0 %	16
Réaliste 30 s	250	13,74 req/s	98,1 ms	0 %	16
Accéléré 1 s	25	25,01 req/s	113,1 ms	0 %	17
Accéléré 1 s	50	45,52 req/s	1 334,6 ms	0 %	26
Accéléré 1 s	100	45,16 req/s	2 604,1 ms	0 %	26

Tableau 7 - Résultats principaux de performance après remédiation

Le scénario réaliste à 250 agents est particulièrement intéressant pour le dimensionnement : aucun échec et un p95 d'environ 98 ms. Le scénario accéléré montre en revanche que la latence augmente lorsque la cadence devient beaucoup plus agressive, même après correction du problème initial de saturation des connexions PostgreSQL. Cette mesure fournit une limite d'ingénierie utile plutôt qu'une promesse de capacité illimitée.

7.4 Télémétrie CPU/GPU et ML

Les tests matériels ont été bornés par des seuils de sécurité. La télémétrie GPU de la machine Windows réelle a été validée avec une NVIDIA GeForce RTX 5070 Laptop GPU : la campagne a

observé environ 65 % d'utilisation, 74 °C au maximum et plus de 4,4 Go de VRAM dans un scénario contrôlé. L'agent a transmis les métriques correspondantes au backend.

La feature GPU a ensuite été intégrée au schéma ML de manière optionnelle afin de ne pas casser les machines sans GPU. La campagne a confirmé la collecte et l'utilisation de la feature, mais n'a pas fabriqué une anomalie GPU lorsque le score réel restait sous le seuil. Ce résultat PARTIAL est méthodologiquement correct : l'absence d'anomalie n'est pas remplacée par une donnée fictive.

7.5 Sécurité

La sécurité du projet couvre plusieurs couches : authentification JWT, refresh HttpOnly côté navigateur, CSRF, RBAC, isolation multi-tenant, tokens agents opaques hashés, audit append-only PostgreSQL, CSP, limitation de taille des payloads, redaction des erreurs, conteneurs non privilégiés et séparation des secrets. PostgreSQL et Redis ne doivent pas être exposés directement au réseau utilisateur.

Les campagnes de sécurité ont également vérifié les dépendances Python et npm, le profil de déploiement Django, les permissions des conteneurs et la protection des communications agent. Les risques résiduels documentés concernent notamment le MFA non implémenté, la nécessité d'un pare-feu egress pour les connecteurs internes et certaines validations de production qui dépendent d'une infrastructure extérieure.

7.6 Bilan de validation

Domaine	Etat	Commentaire
Frontend	PASS	64/64 tests, typecheck/lint/format/build local et Docker.
Backend / API	PASS local	Health, API, persistance, règles, ML et sécurité couverts par la suite de tests.
PostgreSQL / Redis / Celery	PASS local	Services sains, résilience et sauvegarde/restauration validées localement.
WebSocket	PASS	Ticket HTTP 200, handshake 101, reconnexion/replay et sécurité.
Agent Windows réel	PASS	Enrollment, heartbeat, métriques et reconnexion validés.
Performance	PASS/PARTIAL	250 agents réalistes sans erreur ; latence élevée en cadence accélérée.
ML anomalies	PARTIAL	Détection réelle et stabilité améliorée ; vérité terrain industrielle à renforcer.
Prédictif	PARTIAL	Tendances et ETA implémentées ; validation longue durée à poursuivre.
VMware réel	NOT TESTED	Connecteur présent mais aucun vCenter autorisé disponible pendant la campagne.

Domaine	Etat	Commentaire
Hyper-V réel	NOT TESTED	Connecteur présent ; permissions Get-VM insuffisantes lors de la campagne.
SMTP externe	NOT TESTED	Chaîne applicative testée, livraison externe non qualifiée.
Windows Service	NOT TESTED / partiel	Code et scripts présents ; campagne finale à exécuter avec élévation et reboot.

Tableau 8 - Synthèse des validations techniques



Figure 12 - Alertes et recommandations

CHAPITRE 8

Apports du stage, limites et perspectives

8.1 Compétences techniques

Le projet a mobilisé des compétences transversales couvrant systèmes, développement, réseaux, bases de données, DevOps et IA. La mise en place de Docker, PostgreSQL, Redis et Celery a permis de comprendre la différence entre une application monolithique simple et une plateforme répartie en plusieurs services. Les tests de charge ont également montré l'importance du dimensionnement, du pool de connexions et de la mesure des latences.

- Architecture Django REST et séparation des domaines métiers.
- Frontend React/TypeScript et conception d'une interface de supervision NOC.
- Collecte Windows réelle, métriques système et GPU NVIDIA.
- PostgreSQL, Redis, Celery, Beat et traitements asynchrones.
- WebSocket, reconnexion, replay et déduplication d'événements.
- Docker Compose, healthchecks, sauvegarde et restauration.
- Isolation Forest, feature engineering, tendances et limites d'un modèle non supervisé.
- RBAC, multi-tenant, gestion des secrets, audit et sécurité applicative.
- Tests unitaires, intégration, performance, stress contrôlé et validation reproductible.

8.2 Compétences professionnelles

L'apport du stage ne se limite pas aux outils. La principale évolution concerne la manière d'aborder un problème : commencer par observer et auditer, identifier les hypothèses, distinguer les exigences des préférences, construire une preuve, documenter une limite et conserver un état stable. Cette démarche est directement transférable à de futurs projets en entreprise.

Le stage a également renforcé la communication technique. Un rapport utile ne doit pas seulement indiquer qu'un test a échoué ; il doit préciser le contexte, la cause identifiée, l'impact, la correction et le résultat après remédiation. De même, une démonstration doit montrer des données réelles et annoncer explicitement lorsqu'une intégration externe n'a pas pu être validée.

8.3 Difficultés et enseignements

Plusieurs difficultés ont joué un rôle formateur. L'hébergement cloud gratuit s'est révélé peu adapté à une stack nécessitant PostgreSQL, Redis, Celery et WebSocket persistants ; le projet a donc été recentré sur un laboratoire local complet. La saturation initiale de PostgreSQL à 100 connexions a montré qu'une erreur sous charge peut provenir d'une ressource partagée plutôt que du code de l'endpoint lui-même.

La partie ML a également montré qu'un modèle qui produit des anomalies n'est pas automatiquement un bon modèle. Avec peu de fenêtres d'entraînement, les faux positifs deviennent difficiles à interpréter. L'augmentation de l'historique, la stabilité temporelle et l'hystérésis ont donc été plus importantes que l'ajout d'un nouvel algorithme plus complexe.

Enfin, les limites VMware et Hyper-V ont rappelé qu'une intégration réelle dépend autant des droits et de l'environnement que du code. Un connecteur correctement testé en mock ne doit pas être présenté comme validé sur un hyperviseur réel tant que l'accès n'a pas été exécuté dans des conditions autorisées.

8.4 Perspectives

Perspective	Objectif
Hyper-V réel	Exécuter Get-VM avec un compte autorisé, collecter plusieurs VM et valider cycle normal/offline/récupération.
VMware réel	Connecter un ESXi/vCenter de laboratoire, valider découverte, datastore, power state et métriques.
Windows Service	Installer le service, tester démarrage automatique, reboot, reconnexion et désinstallation.
ML long terme	Collecter davantage de données normales, introduire un protocole de labellisation et mesurer précision/rappel/F1.
Prédiction	Valider sur séries réelles longues l'estimation de franchissement de seuil et mesurer l'erreur temporelle.
Scalabilité	Profilier backend et base de données, tester des cadences plus importantes et dimensionner un pooler si nécessaire.
Production	TLS, domaine, reverse proxy, gestion de secrets, CI/CD et monitoring de la plateforme elle-même.
Accessibilité	Compléter audit lecteur d'écran et validation multi-rôle dans plusieurs navigateurs.

Tableau 9 - Limites et perspectives

Conclusion générale

Ce stage d'observation professionnelle réalisé sur site au sein de CGI a permis de relier les enseignements du cycle ingénieur à une démarche de projet plus rigoureuse. L'expérience a donné une vision concrète des exigences d'un environnement professionnel : comprendre le besoin, sécuriser les accès, structurer l'architecture, mesurer les performances, gérer les erreurs, documenter les décisions et ne pas confondre une fonctionnalité développée avec une fonctionnalité réellement validée.

InfraSentinel AI constitue le principal support technique de cette expérience. La plateforme centralise la supervision Windows, prévoit des connecteurs VMware et Hyper-V, utilise PostgreSQL, Redis, Celery et WebSocket et propose un frontend temps réel. Sa couche intelligente combine règles de supervision, détection d'anomalies Isolation Forest et analyse de tendances afin de tendre vers une supervision proactive.

Les résultats locaux sont encourageants : agent Windows réel, télémétrie CPU/GPU, stack Docker saine, frontend entièrement testé, temps réel fonctionnel et capacité démontrée jusqu'à 250 agents réalistes sans erreur dans le scénario mesuré. En parallèle, les limites ont été documentées avec transparence : VMware et Hyper-V réels, certains scénarios Windows Service, SMTP externe et qualification scientifique long terme restent à compléter.

Au-delà du résultat logiciel, le stage a surtout développé une méthode : observer, mesurer, corriger, vérifier et expliquer. Cette méthode représente l'un des acquis les plus importants de l'immersion professionnelle et constituera une base utile pour de futurs projets d'infrastructure, de cybersécurité, de DevOps et d'Intelligence Artificielle.

Bibliographie et webographie

- [1] CGI Maroc - Site institutionnel. <https://www.cgi.com/maroc/fr-fr>
- [2] CGI - Localisations au Maroc et Fès Shore Park. <https://www.cgi.com/en/about-us/locations>
- [3] CGI Maroc - Carrières et environnement professionnel. <https://www.cgi.com/maroc/fr-fr/recrutement>
- [4] Documentation Django. <https://docs.djangoproject.com/>
- [5] Django REST Framework. <https://www.django-rest-framework.org/>
- [6] Django Channels. <https://channels.readthedocs.io/>
- [7] Celery. <https://docs.celeryq.dev/>
- [8] PostgreSQL. <https://www.postgresql.org/docs/>
- [9] Redis. <https://redis.io/docs/>
- [10] scikit-learn - Isolation Forest. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [11] React. <https://react.dev/>
- [12] Vite. <https://vite.dev/>
- [13] Docker. <https://docs.docker.com/>
- [14] OWASP Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>
- [15] Documentation interne du projet InfraSentinel AI. README.md et répertoire docs/ du dépôt de stage

Annexes

Annexe A - Commandes principales de lancement local

Les commandes ci-dessous sont indicatives et doivent être exécutées depuis la racine du dépôt, sans afficher ni recopier de secrets dans le rapport.

```
git status
docker compose build
docker compose up -d
docker compose ps
python manage.py check
python manage.py migrate
npm run typecheck
npm run lint
npm test
npm run build
```

Annexe B - Résultats techniques synthétiques

Indicateur	Résultat de référence
Frontend	17 fichiers / 64 tests, 64 PASS, 0 FAIL, 0 SKIP
TypeScript / ESLint / Prettier	PASS
Build frontend	PASS local et Docker
npm audit	0 vulnérabilité connue lors de la campagne
WebSocket	Ticket HTTP 200, handshake 101, reconnexion/replay couverts
Charge réaliste	250 agents, 0 % erreur, p95 98,1 ms
PostgreSQL après remédiation	26 connexions max en charge accélérée étudiée, 0 erreur aux paliers rejoués
GPU réel	Environ 65 % utilisation max, 74 °C max, plus de 4,4 Go de VRAM dans le scénario contrôlé
Limites externes	VMware réel, Hyper-V réel, SMTP externe et certains scénarios service/reboot non qualifiés

Annexe C - Vérification des captures intégrées avant remise

- Vue globale : vérifier qu'aucune donnée simulée n'est présentée comme réelle.
- Machines : afficher au moins la machine Windows réellement supervisée et son état.
- Fiche machine : CPU, RAM, disque, réseau, GPU/VRAM si disponibles, historique et dernier heartbeat.
- Alertes : privilégier une alerte issue d'un test contrôlé clairement identifié.
- Anomalies ML : montrer le modèle et une explication compréhensible, sans exposer de secret.
- Prédictions : afficher la tendance et la formulation prudente du risque.

14. Agents : masquer le token, les secrets, les chemins privés ou informations non destinées à la diffusion.
15. Avant export final : relire chaque capture pour retirer adresses, e-mails ou données confidentielles inutiles.

Annexe D - Note sur la validation réelle et contrôlée

Règle de présentation au jury

Une preuve simulée sert à valider la scalabilité ou la reproductibilité d'un scénario. Elle ne remplace pas une preuve d'intégration réelle. Le rapport et la démonstration doivent conserver les libellés REAL PASS, CONTROLLED TEST PASS, SIMULATED LOAD TEST PASS et NOT TESTED lorsque ces distinctions sont pertinentes.

Fin du rapport